Python Engineering - Reference Guide

Class 08 - Data Processing with NumPy

Quick guide to boolean masks, filtering, statistics, and common NumPy data-processing operations.

1. Boolean mask

```
mask = heights > 100
# [False, False, True, True, ...]
```

A mask is True where the condition is satisfied and False elsewhere.

2. Filtering

```
selected_values = heights[mask]
selected_rows = trajectory_array[mask]
```

3. Combining conditions

Operation	NumPy	Example
AND	&	(h > 50) & (v > 0)
OR	|	(h > 100) | (v < -40)
NOT	~	~(v > 0)

Individual conditions should normally be wrapped in parentheses.

4. Counting True values

```
count = np.sum(mask)
# True behaves like 1, False like 0
```

5. np.where()

```
indices = np.where(mask)[0]
```

Returns positions where a condition is True. For 1D data we commonly take [0].

6. Basic statistics

Function	Meaning
np.mean(x)	mean
np.std(x)	standard deviation
np.min(x)	minimum
np.max(x)	maximum
np.median(x)	median
np.sum(x)	sum

7. Consecutive differences

```
dx = np.diff(x)
# dx[i] = x[i+1] - x[i]
```

Length changes from N to N-1.

8. Adding columns

```
processed = np.column_stack((
    heights,
```

```
    velocities,
    times,
    energy
))
```

np.column_stack groups 1D vectors as columns of a 2D matrix.

9. Range selection

```
mask = (times >= 2.0) & (times <= 5.0)
window = trajectory_array[mask]
```

10. Approximate equality

```
np.allclose(dt_samples, config['dt'])
```

For floating-point data, tolerance-based comparison is usually better than exact ==.

11. Mental pattern

```
QUESTION
   -> CONDITION
      -> BOOLEAN MASK
         -> FILTER / COUNT / INDICES
            -> STATISTIC / ENGINEERING RESULT
```

Mini cheat sheet

```
mask = x > threshold
x[mask]                      filter values
A[mask]                      filter rows
np.sum(mask)                 count True
np.where(mask)[0]            indices
(c1) & (c2)                  AND
(c1) | (c2)                  OR
~c1                          NOT
np.mean(x), np.std(x)         stats
np.diff(x)                   consecutive differences
np.column_stack((a,b,c))      add columns
np.allclose(a,b)              approximate equality
```